pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: A First Look at the Data

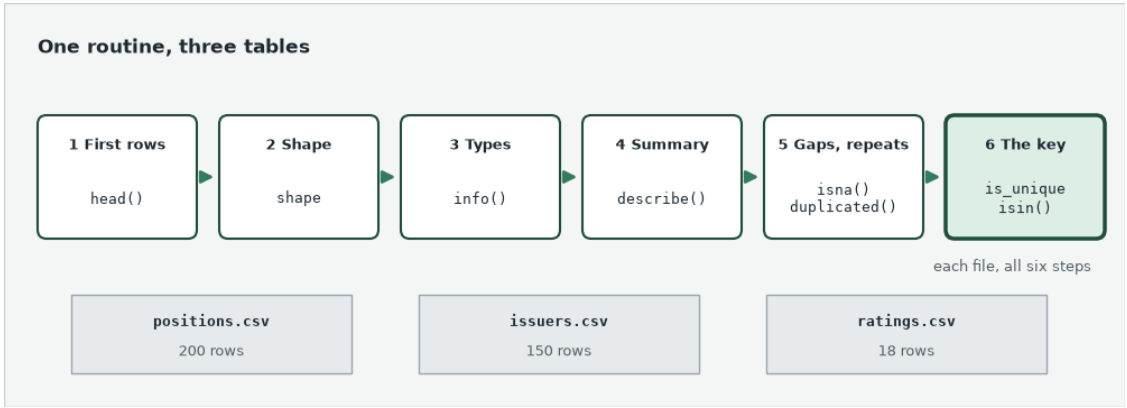
WEEK 2, DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's fourteenth note

Yesterday we checked the joins; nobody looked at each file on its own. The three systems now send these files every month, and nobody tells us what is wrong with them. Before tomorrow's report, tell me what each file holds, whether anything is missing or doubled, and that the face still ties to our 489,000,000. Give me the bonds to watch: below par, long-dated, small. Will the three join without losing or doubling a bond? And write the checks so they run themselves next month.




```
positions = pd.read_csv(data_folder + 'positions.csv')
issuers = pd.read_csv(data_folder + 'issuers.csv')
ratings = pd.read_csv(data_folder + 'ratings.csv')
print(positions.shape, issuers.shape, ratings.shape)
(200, 6) (150, 5) (18, 4)
```

As of 2026-09-30; the issuers and CUSIPs are invented.

One rating per issuer here; in real data a bond can sit a notch away from its issuer.

Look first, join tomorrow: the same six steps for every table.

Three sizes, so a row stands for something different in each table. Find out what.

9.2 One row, one bond held; the maturity is text

```
positions.head()
```

	cusip	ticker	coupon	maturity	price	par_held
0	99080CAE8	BEZE	7.000	2032-06-18	106.727	5000000
1	99049XAE2	BEZH	8.125	2029-04-17	97.556	250000
2	99039NAC0	BEZL	5.125	2030-06-03	98.730	1500000
3	99039NAG1	BEZL	6.125	2053-04-09	92.261	500000
4	99034IAD4	BEZN	4.625	2036-04-05	92.883	2500000

The CUSIP names the bond; the ticker names its issuer, and BEZL already appears twice.

One row, one bond held: the CUSIP names it.

```
positions.info()
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   cusip     200 non-null    str
 1   ticker    200 non-null    str
 2   coupon    200 non-null    float64
 3   maturity  200 non-null    str
 4   price     200 non-null    float64
 5   par_held  200 non-null    int64
dtypes: float64(2), int64(1), str(3)
memory usage: 14.0 KB
```

```
maturity_dates = pd.to_datetime(positions['maturity'])
as_of = pd.Timestamp('2026-09-30')
print('First:', maturity_dates.min().date(), 'Last:', maturity_dates.max().date())
print('Matured already:', (maturity_dates <= as_of).sum())
First: 2027-10-02 Last: 2056-08-23
Matured already: 0
```

Every column holds 200 non-null values, and non-null means not missing. "maturity" is "str", text, as Combining Part 2 found in the holdings.

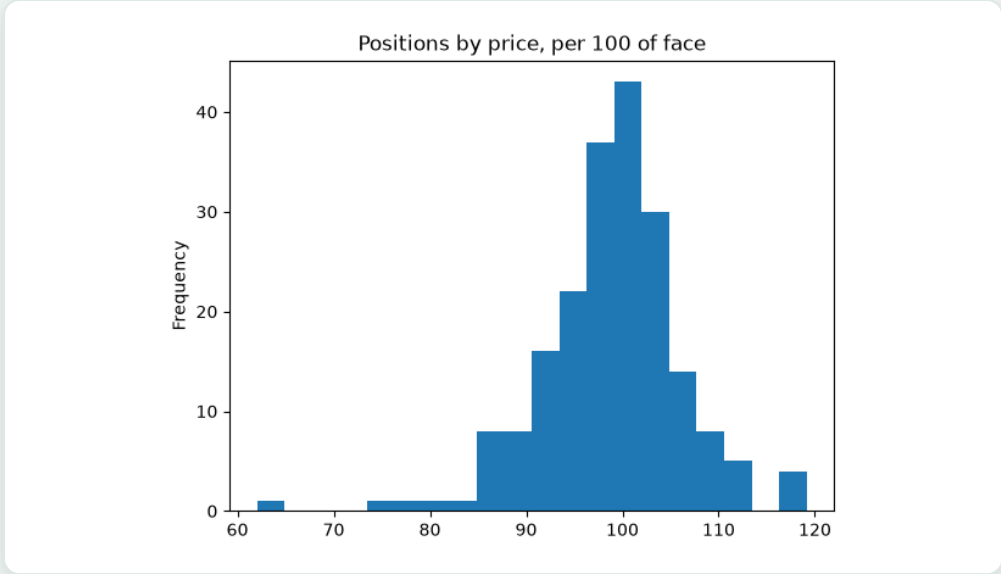
Numbers ready; the maturity is text, converted on the side.

The range of the numbers, and one price far below

```
positions.describe()
```

	coupon	price	par_held
count	200.000000	200.000000	2.000000e+02
mean	5.937500	98.828295	2.445000e+06
std	1.648435	7.404931	1.366818e+06
min	3.250000	62.045000	2.500000e+05
25%	4.750000	95.140000	1.500000e+06
50%	5.687500	99.416000	2.250000e+06
75%	6.875000	103.004000	3.500000e+06
max	11.000000	119.131000	5.000000e+06

The three text columns are left out, with no message. "2.445000e+06" is scientific notation, as in week 1's Day 5: the average position is 2,445,000 of face. A hill from about 93 to 102 around the mean of 98.83, and one bar alone at the far left, three empty bins after it.




```
positions.nsmallest(3, 'price')
```

	cusip	ticker	coupon	maturity	price	par_held
17	99009JAA9	CLDB	9.500	2034-12-06	62.045	3000000
117	99054CAG5	NEVX	3.875	2048-01-06	73.873	1000000
150	99053BAB9	SYLX	3.625	2055-09-03	76.903	4500000

CLDB at 62.045, nearly 12 points under the next: the same three pandas Part 2 found in the holdings file.

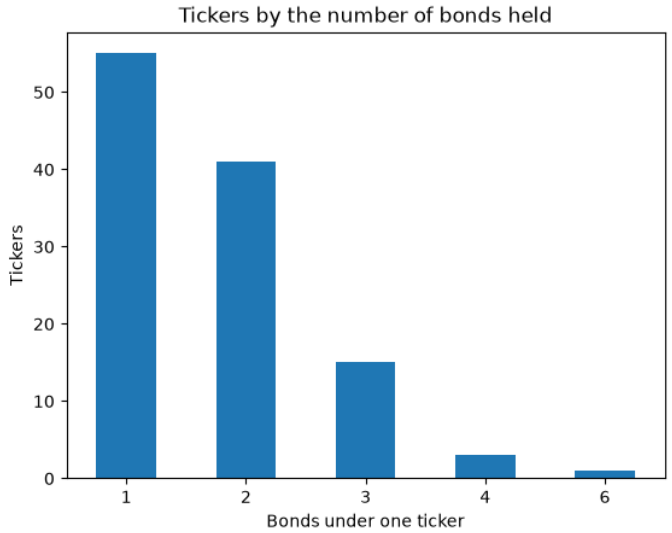
One price sits far below the rest: CLDB. The files show it; they cannot say why, so the analyst asks the desk.

Nothing missing, nothing doubled; the face ties

```
print(positions.isna().sum())
print('Repeated rows:', positions.duplicated().sum())
cusip       0
ticker      0
coupon      0
maturity    0
price       0
par_held    0
dtype: int64
Repeated rows: 0
```

```
total_face = positions['par_held'].sum()
print('Total face held:', total_face)
Total face held: 489000000
```

A missing price would not stop a total: pandas skips it, and the total comes out too small. A position sent twice would double its face. pandas Part 2 summed the same 489,000,000 from "holdings.csv", a different file.



55 tickers have one bond and 60 have more; the lone bar at 6 is NEVX. One ticker, six CUSIPs, maturing from 2031 to 2051.

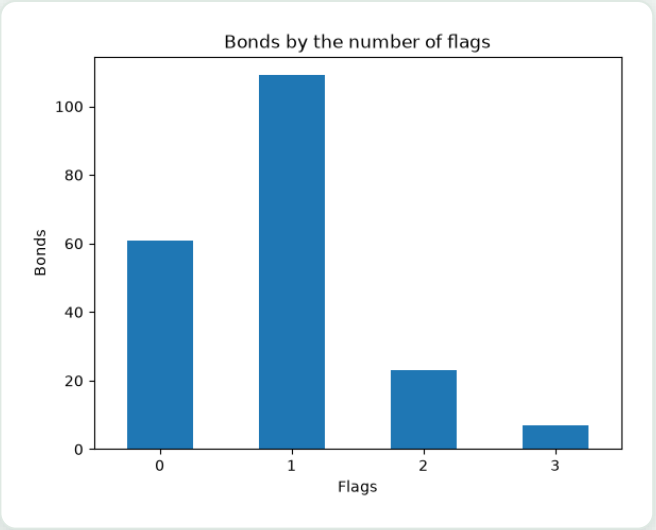
Nothing missing, nothing doubled, and two systems agree on 489,000,000.

The ticker repeats; only the CUSIP tells every bond apart.

Bonds to watch: three flags, counted down and across

```
flags = pd.DataFrame({
    'below_par': positions['price'] < 100,
    'long_dated': maturity_dates > pd.Timestamp('2040-12-31'),
    'small': positions['par_held'] < 1000000,
})
flags.sum()
below_par     113
long_dated     34
small          29
dtype: int64
```

```
flag_count = flags.sum(axis=1)
bonds_by_flags = flag_count.value_counts().sort_index()
chart = bonds_by_flags.plot.bar(rot=0, title='Bonds by the number of flags',
                                xlabel='Flags', ylabel='Bonds')
```



The 113 below par is pandas Part 2's count. 61 bonds carry no flag, 109 one, 23 two, and 7 all three.

Seven bonds trip all three tests. The list describes the file; it judges no bond.

Excel side by side. Price in column E, the three flags in helper columns G to I

Job	pandas	Excel
Size	`positions.shape`	Ctrl+End, the last used cell;
		a column's status-bar Count, 201 with the header
Gaps	`positions['price'].isna().sum()`	=COUNTBLANK(E2:E201)
Repeats	`positions.duplicated().sum()`	Data, Remove Duplicates, which deletes the copies and says how many
Flags	`positions['price'] < 100, flags.sum(axis=1), flags.sum()`	=IF(E2<100, 1, 0) in G, =SUM(G2:I2), =SUM(G2:G201)

9.3 One row per issuer; a rating is letters

```
print(issuers.shape)
issuers.head()
(150, 5)
```

	ticker	issuer	sector	country	rating
0	QVNE	Quorvane Energy	Energy	US	BBB
1	DNMR	Dunmarrow Rail	Transportation	US	BB+
2	PLWK	Pellwick Paper	Materials	US	A-
3	HLVF	Halvering Foods	Consumer	US	B+
4	OSTV	Ostrevale Telecom	Telecom	US	BBB-

One row is one issuer, with no bond in it. The two files run in different orders, so they match on "ticker", the one column they share.

```
issuers[issuers['ticker'] == 'NEVX']
```

	ticker	issuer	sector	country	rating
53	NEVX	Nevulex Health	Health Care	US	BBB-

```
print(issuers['sector'].value_counts())
print(issuers['country'].unique().tolist())
sector
Materials         18
Utilities         18
Energy            17
Industrials       17
Telecom           16
Transportation    15
Consumer          14
Financials        14
Health Care       12
Technology         9
Name: count, dtype: int64
['US']
```

Five text columns, 150 values each, no repeat; an issuer's gap would land on every one of its bonds. One country, so a report by country would have a single row.

```
issuers['rating'].mean()
TypeError: Cannot perform reduction 'mean' with string dtype
```

Nevulex Health, Health Care, BBB-: one row behind six bonds.

A rating is letters. Its number, the score, is in the third table.

9.4 The whole scale: 18 notches, two grades

	rating	score	bucket	grade
0	AAA	1	AAA	Investment Grade
1	AA+	2	AA	Investment Grade
2	AA	3	AA	Investment Grade
3	AA-	4	AA	Investment Grade
4	A+	5	A	Investment Grade
5	A	6	A	Investment Grade
6	A-	7	A	Investment Grade
7	BBB+	8	BBB	Investment Grade
8	BBB	9	BBB	Investment Grade
9	BBB-	10	BBB	Investment Grade
10	BB+	11	BB	High Yield
11	BB	12	BB	High Yield
12	BB-	13	BB	High Yield
13	B+	14	B	High Yield
14	B	15	B	High Yield
15	B-	16	B	High Yield
16	CCC+	17	CCC	High Yield
17	CCC	18	CCC	High Yield

```
print(ratings.dtypes)
ratings
rating      str
score     int64
bucket      str
grade       str
dtype: object
```

One row is one rating, AAA at score 1 down to CCC at 18; "score" is the number the mean lacked.

```
print('Scores 1 to 18:', ratings['score'].tolist() == list(range(1, 19)))
print('Missing:', ratings.isna().sum().sum(), 'Repeated:', ratings.duplicated().sum())
ratings.groupby('grade')['score'].agg(['count', 'min', 'max'])
Scores 1 to 18: True
Missing: 0 Repeated: 0
```

grade	count	min	max
High Yield	8	11	18
Investment Grade	10	1	10

18 notches, two grades that neither overlap nor leave a gap, nothing missing: a complete scale.

9.5 Is each key unique, and can a test fail?

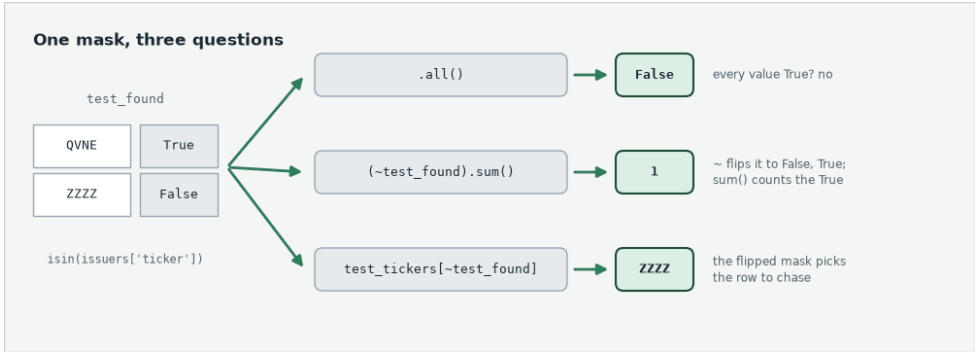
```
print('cusip in positions:', positions['cusip'].is_unique)
print('ticker in issuers:', issuers['ticker'].is_unique)
print('rating in ratings:', ratings['rating'].is_unique)
print('ticker in positions:', positions['ticker'].is_unique)
cusip in positions: True
ticker in issuers: True
rating in ratings: True
ticker in positions: False
```

Table	Column	Unique	Role
positions	`cusip`	yes	names one bond
positions	`ticker`	no	points to an issuer
issuers	`ticker`	yes	names one issuer
issuers	`rating`	no	points to a rating
ratings	`rating`	yes	names one rating

Many on one side, one on the other: the shape Combining Part 1's "validate='many_to_one'" demands.

```
test_tickers = pd.Series(['QVNE', 'ZZZZ'])
test_found = test_tickers.isin(issuers['ticker'])
test_found
0     True
1    False
dtype: bool
```

```
print(test_found.all())
print((~test_found).sum())
test_tickers[~test_found]
False
1
1    ZZZZ
dtype: str
```



9.6 The findings as checks, and one that fails

```
assert positions.isna().sum().sum() == 0, 'the positions have missing values'
assert positions.duplicated().sum() == 0, 'the positions have repeated rows'
assert issuers['ticker'].is_unique, 'a ticker appears twice in the issuers table'
assert ratings['rating'].is_unique, 'a rating appears twice on the scale'
assert positions['ticker'].isin(issuers['ticker']).all(), 'a ticker is missing from issuers'
assert issuers['rating'].isin(ratings['rating']).all(), 'a rating is not on the scale'
print('Six checks passed:', positions.shape, issuers.shape, ratings.shape)
```

```
assert positions['ticker'].is_unique, 'the ticker repeats, so it cannot identify a bond'
AssertionError: the ticker repeats, so it cannot identify a bond
```

Excel side by side. A check cell, "=IF(COUNTIF(G2:G201, FALSE)=0, "OK", "CHECK")", shows its result while every other formula calculates on; an "assert" stops the run.

Today a person read every True and count; next month nobody will. The message is for someone months from now: say what was expected and what it means.

The findings now run on next month's files, and stop at the first one that fails.

Excel side by side. Positions sheet: CUSIP in A, ticker in B, a helper in G; an "issuers" sheet with the ticker in A

Job	pandas	Excel
Unique key	`positions['cusip'].is_unique`	=COUNTA(UNIQUE(A2:A201))=COUNTA(A2:A201), Excel 2021 and later and Microsoft 365
Found	`positions['ticker'].isin(issuers['ticker'])`	=COUNTIF(issuers!A:A, B2)>0 in G
All found	`found.all()`	=AND(G2:G201)
Flip, count	`~found, (~found).sum()`	=NOT(G2), =COUNTIF(G2:G201, FALSE)

Predict first: decide, then run each in the notebook

Predict 1

The next cell runs "describe()" on the three text columns, "cusip", "ticker", and "maturity". What comes back? A) An error: text has no mean. B) An empty table. C) A different summary, one that counts.

```
positions[['cusip', 'ticker', 'maturity']].describe()
```

Predict 3

The issuers carry 18 distinct ratings over 150 rows, and the scale lists 18. What does "issuers['rating'].isin(ratings['rating']).all()" return? A) True. B) False: 150 is not 18. C) 18.

```
rating_found = issuers['rating'].isin(ratings['rating'])
print('Every rating found:', rating_found.all())
found = positions['ticker'].isin(issuers['ticker'])
not_found = ~found
print('Every ticker found:', found.all(), 'Not found:', not_found.sum())
```

Predict 2

The next cell stacks the first three rows with a second copy of the third, four rows in all, and runs "duplicated()". What does it mark? A) False, False, True, True: both copies. B) False, False, False, True: the second copy only. C) A single True: the table holds a repeat.

```
first_three = positions.head(3)
with_repeat = pd.concat([first_three, first_three.tail(1)], ignore_index=True)
with_repeat.duplicated()
```

Predict first: the issuers left out, the shape after the join

Predict 4

The positions hold 115 distinct tickers, every one found among the 150 issuers. How many issuers have no bond in the positions? Write down a number.

```
held = issuers['ticker'].isin(positions['ticker'])
not_held = issuers[~held]
print('Issuers held:', held.sum(), 'Not held:', len(not_held))
not_held.head(3)
```

Predict 5

Tomorrow the positions (6 columns) join the issuers (5) on "ticker", then the scale (4) on "rating", each key kept once. How many columns will the final table have? Write down a number.

```
rows = positions.shape[0]
columns_after_first = positions.shape[1] + issuers.shape[1] - 1
columns_after_second = columns_after_first + ratings.shape[1] - 1
print('After issuers:', (rows, columns_after_first))
print('After ratings:', (rows, columns_after_second))
```

The note, answered

	positions	issuers	ratings
One row is	one bond held	one issuer	one rating
Rows, columns	200, 6	150, 5	18, 4
Missing, repeated	0, 0	0, 0	0, 0
Unique key	cusip	ticker	rating

The note asked	Answer
What each file holds	one bond held, one issuer, one rating: 200, 150, 18 rows
Missing or doubled	nothing, in any of the three
The face ties	489,000,000, as in the holdings file
Bonds to watch	113 below par, 34 long-dated, 29 small; 7 all three
Checks that run themselves	six assert lines

Tomorrow the three tables become one book.